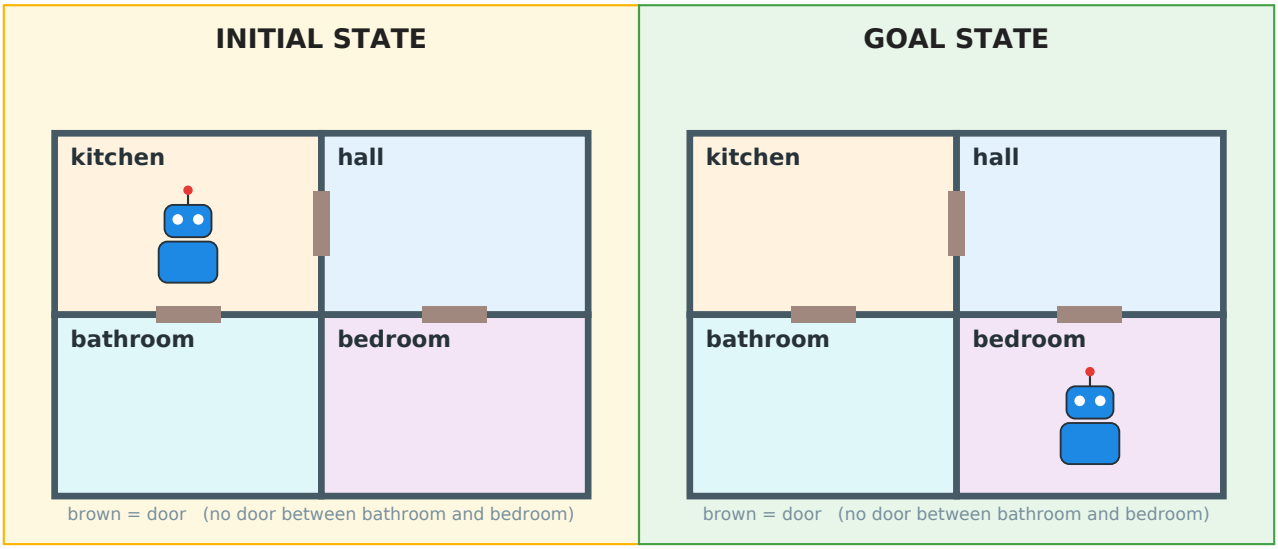
Problem 1: Robot Navigation

Level: **Very basic** · Folder: **01_Robot_Navigation/** · Files: domain.pddl, problem.pddl, solution_plan.txt

1. The problem

A household robot stands in the **kitchen** of a small house with four rooms: kitchen, hall, bathroom and bedroom. Some rooms are joined by doors; the robot can only move through a door into a neighbouring room. There is **no door** between the bathroom and the bedroom. Find a sequence of moves that brings the robot to the **bedroom**.



The goal picture shows one state that satisfies the goal (the final state of the plan). Only the facts in the :goal section are required; anything else may be different.

What you will learn

- The two-file structure of PDDL: a **domain** (general rules) and a **problem** (one specific task).
- **:types** and typed **:objects**.
- **Predicates**: facts that are either true or false in a state.
- An **action** with parameters, a precondition and an effect (add and delete lists).
- The difference between **static** facts (never change) and **fluent** facts (changed by actions).

2. Formulating the problem in PDDL

Step 1: Types and objects. One type `room`; four objects: kitchen, hall, bedroom, bathroom.

Step 2: Predicates describe what can be true or false in a state:

Predicate	Meaning
`(robot-at ?r)`	The robot is currently in room ?r. *Fluent* (changed by move).
`(connected ?from ?to)`	A door leads from ?from to ?to. *Static* (no action changes it).

Step 3: Actions. Each action has *parameters*, a *precondition* (what must be true to apply it) and an *effect* (facts that become **true (+)** or **false (−)**):

Action	Precondition	Effect
move `?from - room, ?to - room` Walk from room ?from to room ?to. Allowed only if the robot is in ?from and a door connects the two rooms. Afterwards the robot is in ?to and **no longer** in ?from.	`(robot-at ?from)` `(connected ?from ?to)`	`+ (robot-at ?to)` `− (robot-at ?from)`

Step 4: Initial state (`:init`) lists *every* fact that is true at the start. Everything else is false (closed-world assumption). **Step 5: Goal** (`:goal`) lists the facts that must be true at the end.

INITIAL STATE (7 facts)	GOAL

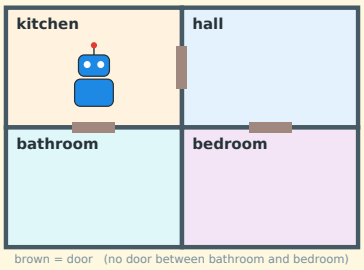
`(robot-at kitchen)`
`(connected kitchen hall)`
`(connected hall kitchen)`
`(connected hall bedroom)`
`(connected bedroom hall)`
`(connected kitchen bathroom)`
`(connected bathroom kitchen)`
	`(robot-at bedroom)`

Modelling notes

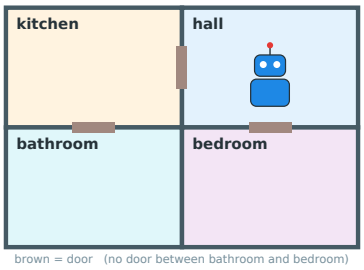
- PDDL facts have no built-in symmetry: `(connected kitchen hall)` does not imply `(connected hall kitchen)`. Because doors work both ways, both directions are listed in **:init**.
- The delete effect `(not (robot-at ?from))` is essential. Without it the robot would be in two rooms at once after moving.
- Anything not listed in **:init** is false (the *closed-world assumption*). So bathroom-bedroom has no door simply because we did not write one.
- The planner cannot go directly from the bathroom to the bedroom, and the shortest route is kitchen, hall, bedroom (2 moves).

3. Solution plan (2 actions, optimal)

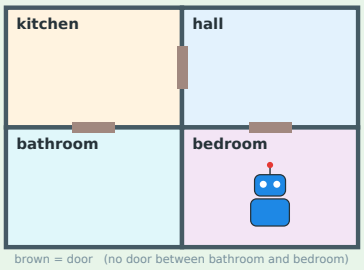
Each line of the plan is one **ground action**: the action name with every parameter replaced by an object. The pictures below show the state **after** each action. They were produced by executing the plan on the PDDL model.



Initial state



Step 1
`(move kitchen hall)`



Step 2
`(move hall bedroom)`

4. How to run it

Online (no installation)	Open **editor.planning.domains**, paste *domain.pddl* and *problem.pddl* into two tabs and press **Solve**.
pyperplan (Python)	`pip install pyperplan` `pyperplan domain.pddl problem.pddl` (breadth-first search) `pyperplan -s astar -H lmcut domain.pddl problem.pddl` (optimal A*)
Fast Downward	`fast-downward.py domain.pddl problem.pddl --search "astar(lmcut())"`

5. Lab exercises

1. Add a door between the bathroom and the bedroom. Does the length of the plan change? Why or why not?
2. Add a fifth room *garage*, connected only to the bathroom, and make it the goal.
3. Delete only `(connected hall bedroom)` (keep the reverse direction). What does the planner report?
4. Add a predicate `(visited ?r)` that **move** makes true, and set the goal to visiting both the bathroom and the bedroom.

6. PDDL source code (appendix)

domain.pddl

```
;; ============================================================
;; Problem 1 : Robot Navigation
;; Level     : Very basic
;; A robot moves between rooms that are connected by doors.
;; ============================================================
(define (domain robot-navigation)
  (:requirements :strips :typing)

  (:types room)

  (:predicates
    (robot-at ?r - room)                 ; the robot is in room ?r
    (connected ?from - room ?to - room)  ; a door leads from ?from to ?to
  )

  ;; Move the robot from one room to a connected room
  (:action move
    :parameters (?from - room ?to - room)
    :precondition (and (robot-at ?from)
                       (connected ?from ?to))
    :effect (and (robot-at ?to)
                 (not (robot-at ?from)))
  )
)
```

problem.pddl

```
;; Robot starts in the kitchen and must reach the bedroom.
;;
;;   +---------+---------+
;;   | kitchen | hall    |
;;   +---------+---------+
;;   | bathroom| bedroom |
;;   +---------+---------+
;; Doors: kitchen-hall, hall-bedroom, kitchen-bathroom
(define (problem robot-nav-1)
  (:domain robot-navigation)

  (:objects kitchen hall bedroom bathroom - room)

  (:init
    (robot-at kitchen)
    ;; doors work in both directions, so we list both
```

```
    (connected kitchen hall)     (connected hall kitchen)
    (connected hall bedroom)     (connected bedroom hall)
    (connected kitchen bathroom) (connected bathroom kitchen)
  )

  (:goal (robot-at bedroom))
)
```

solution_plan.txt

```
1. (move kitchen hall)
2. (move hall bedroom)
```